



Implementation Plan

Personalized Gym Web App

Execution roadmap for PRD + TRD + Concept 3 UI

MVP -> Private Beta -> Production V1

Recommended delivery model: 9 two-week sprints

Bodybuilding

Calisthenics

Athletic

Aesthetic

Nutrition

Computer Vision

Analytics

Implementation principle

Ship deterministic tracking first. Add adaptive AI only after clean user data exists.

Keep CV and recommendations versioned, explainable, privacy-aware, and feature-flagged.

01. Executive Implementation Summary

A delivery-oriented plan for building AthleteOS from repository bootstrap to production V1.

Recommended target: a production-capable V1 in 18 weeks (9 two-week sprints) for a focused 5-7 person product team. A smaller team can use the same sequence with a longer calendar.

The implementation is intentionally staged. Authentication, onboarding, workout logging, nutrition tracking, habit tracking, and analytics create the clean longitudinal data required for trustworthy personalization. CV and adaptive recommendations are integrated early behind feature flags, but are not allowed to become hard dependencies for core usage.

Primary release objectives

- V1 usability: an athlete can onboard, receive a plan, log every set, track meals/water/habits, and view weekly/monthly progress.
- Personalization: training type, experience level, equipment, performance history, adherence, recovery, and optional CV proportions influence recommendations.
- Reliability: core workout logging works on weak networks with conflict-safe sync.
- Privacy: progress media is optional; browser-side CV is preferred; raw images can be discarded when the user selects analysis-only mode.
- Scalability: modular monolith first with service boundaries that allow later separation.

Definition of project success

Area	V1 success condition
Onboarding	Median completion under 5 minutes; athlete profile and goal are persisted.
Workout	User can log sets/reps/load/RIR; resume interrupted sessions; PRs and volume calculate correctly.
Nutrition	Fast search for Indian and international foods; servings and macros are auditable.
Habits	Spreadsheet-like month view with schedule-aware streaks.
Analytics	Weekly/monthly comparisons update from event data without manual reconciliation.
CV	Optional scan returns quality/confidence + proportions/posture indicators; no medical diagnosis claims.
Operations	Automated CI, migrations, monitoring, backups, security tests, feature flags, rollback path.

02. Scope, Constraints and Build Principles

In-scope for production V1

- Responsive web/PWA experience using Concept 3 - Vibrant & Bold design language.
- Email/phone-ready authentication architecture, profile management, consent and account deletion.
- Athlete onboarding, body metrics, goals, training type, experience, equipment and recovery baseline.
- Optional progress-photo/body-analysis flow.
- Food search, meals, water, nutrition targets, recipes/servings.
- Workout plan generation, day/session logging, rest timer, notes, PR and progression tracking.
- Excel-style habits, streaks, reminders model, weekly/monthly reports.
- Admin console for catalogues, moderation, feature flags and support tooling.

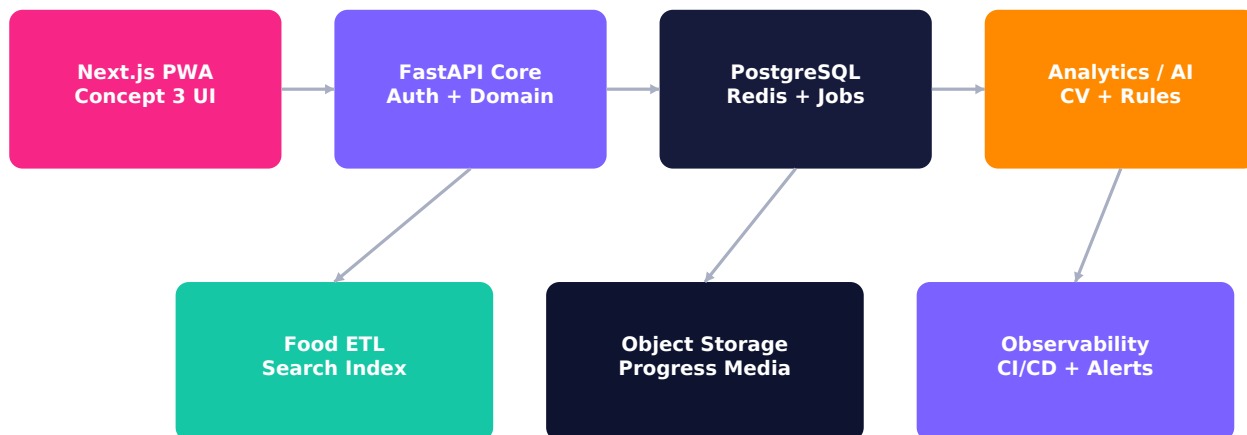
Explicitly deferred from V1

- Real-time exercise form correction from live video.
- Medical body-fat estimation or medical/nutritional diagnosis.
- Coach marketplace, social feed, payments/subscriptions, smart-watch integrations.
- End-to-end generative AI coach with unconstrained plan generation.
- Native iOS/Android apps; PWA architecture should keep this path open.

Engineering principles

Principle	Implementation rule
Modular monolith first	Keep auth, athlete, workout, nutrition, habit and analytics modules isolated in code and schema. Extract services only when load/team boundaries justify it.
Deterministic before ML	Progression and targets use testable rules. ML/ranking may modify suggestions later but never hide base reasoning.
Event-aware data model	Every important user action emits an analytics/domain event; reports never depend on UI-side calculation.
Offline-friendly workout UX	Session draft state is locally durable; API uses idempotency keys and versioned updates.
Feature flags	CV, adaptive plan changes and experimental scoring can be disabled without redeploying.
Privacy by default	Collect only required data; segregate private media; short-lived upload URLs; user controls retention.

03. Target Technical Architecture



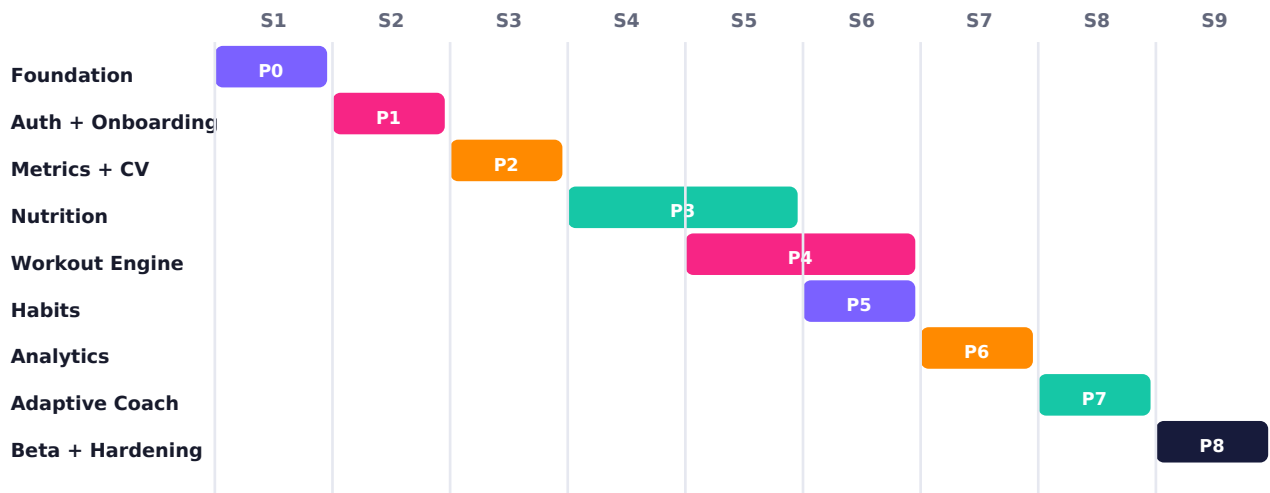
Recommended baseline stack

Layer	Technology / approach	Why
Frontend	Next.js + TypeScript + component library + PWA service worker	Fast product iteration, strong routing/SSR options, mobile-capable installable UX.
Backend	FastAPI + Python modular monolith	Pairs cleanly with CV/ML code and supports typed API contracts.
Primary data	PostgreSQL	Strong relational integrity for athlete/workout/nutrition/habit histories.
Cache/jobs	Redis + worker queue	Caching, idempotency, rate limit state, ingestion and report jobs.
Media	S3-compatible object storage	Private progress media with signed URLs and lifecycle policies.
CV	MediaPipe Pose / segmentation adapter + Python fallback	Pose/proportion extraction without coupling product logic to one model.
Analytics	Domain events -> aggregate tables/materialized views	Fast weekly/monthly reports with auditable raw data.
Delivery	Containerized API/worker + managed DB + CI/CD	Predictable builds, migrations, rollback and observability.

Repository structure

apps/web - Next.js UI apps/api - FastAPI workers - ingestion/reports packages/ui - design system packages/contracts - generated/openapi types ml/cv - CV adapter infra - IaC/deployment docs - ADRs/runbooks

04. Delivery Model and Sprint Roadmap



Sprint	Primary outcome	Release gate
S1	Foundation, environments, design system, auth skeleton	CI green; staging deployed; schema migration works.
S2	Registration/login, onboarding, athlete profile	New user reaches personalized dashboard shell.
S3	Metrics, measurements, progress photos, CV alpha	Scan optional; metrics timeline works without CV.
S4	Nutrition catalogue, search, meal/water logging	Indian dish search + serving macros pass test corpus.
S5	Nutrition polish + workout catalogue/plan model	Daily calories/macros accurate; plan data model frozen.
S6	Workout sessions, set logging, PR/progression + habits	Day view and offline session recovery complete.
S7	Habit month grid + reports/analytics	Weekly/monthly report matches source events.
S8	Adaptive plan rules, readiness, recommendations	Suggestion reasoning visible; override/undo supported.
S9	Security, performance, beta fixes, production release	Release checklist, monitoring, rollback and backups verified.

Release train

- Internal Alpha: end of Sprint 3 - onboarding, metrics, CV alpha.
- Functional Alpha: end of Sprint 6 - nutrition + workouts + habits.
- Private Beta: end of Sprint 8 - analytics + adaptive suggestions.
- Production V1: end of Sprint 9 after hardening and operational readiness.

05. Phase 0 - Product and Engineering Foundation

Goal: create the delivery platform once so feature teams do not repeatedly solve environments, UI primitives, logging, migrations, and release mechanics.

Workstream	Tasks	Exit criteria
Repository	Monorepo; lint/format/test conventions; branch protection; CODEOWNERS; ADR template.	Fresh clone runs web + API locally with one documented command.
Design system	Tokens for dark navy, pink/orange/purple gradients; typography; spacing; cards; buttons; inputs; charts; empty/loading/error states.	Concept 3 dashboard primitives reusable across pages.
API base	FastAPI app, error envelope, request IDs, auth middleware hooks, OpenAPI generation.	Contract test from web client passes.
Database	Migration framework; local/staging DB; base user/consent/audit tables.	Up/down migration tested in CI.
Observability	Structured logs, error tracking, metrics, health/readiness endpoints.	Synthetic health check visible in staging.
CI/CD	Build/test/security scan; staging deploy; migration job; smoke tests.	Failed smoke test blocks promotion.

Phase 0 backlog

- Create environment matrix: local, test, staging, production.
- Set secrets policy and rotation ownership.
- Seed demo athlete and deterministic fixtures.
- Create feature flags: cv_scan, adaptive_recommendations, nutrition_external_ingest, beta_reports.
- Define event names and analytics payload conventions before UI tracking starts.

06. Phase 1 - Authentication and Athlete Onboarding

User flow

Register -> verify/login -> consent -> basic profile -> body metrics -> lifestyle/recovery baseline -> fitness level -> training type -> equipment -> goals -> dashboard.

Implementation tasks

Area	Backend / data	Frontend / UX	Acceptance
Identity	User, auth identity, sessions, password/reset hooks, refresh/session revocation.	Login/register/forgot-password forms; validation; loading/error states.	Unauthorized APIs reject; session expiry/re-login tested.
Consent	Terms/privacy version, optional media/CV consent, audit timestamp.	Separate required product consent from optional photo analysis consent.	Consent changes are auditable.
Athlete profile	DOB/age range where appropriate, units, height, weight, goals, training type, level, equipment.	Wizard with progress indicator and autosave.	Resume onboarding from last completed step.
Lifestyle	Water, sleep, activity, training days, session length, injury limitation note field.	Accessible controls; metric/imperial conversion.	Canonical units stored; display conversion tested.
Initial plan shell	Generate plan template reference and nutrition target placeholders.	Dashboard loads real profile and next actions.	No empty dashboard after onboarding.

Data quality rules

- Store canonical weight in kg and length in cm; keep preferred display unit separately.
- Use range validation and soft warnings for implausible values rather than silently accepting bad metrics.
- Every profile update includes changed_at and source (manual/import/system).
- Do not infer gender/sex/body type from images.

07. Phase 2 - Body Metrics and Computer Vision

CV objective: assist with standardized progress assessment using pose landmarks, silhouette/segmentation, visible proportions, posture indicators, symmetry and image quality. The output is a fitness progress signal, not a medical diagnosis.

Stage	Implementation	Fallback / safety
Capture	Front + optional side photo instructions; camera distance/lighting/pose guide; consent check.	Manual metrics remain fully usable if user skips photos.
Quality gate	Check person count, crop, visibility, pose completeness, blur/exposure proxy.	Return actionable retake message; never produce confident score on low-quality input.
Inference	Browser-first MediaPipe adapter; optional server inference behind signed upload and short retention.	Feature flag can disable server path independently.
Features	Shoulder/waist/hip ratios where visible; left-right alignment; torso/limb landmarks; posture deltas; scan confidence.	Missing landmarks stored as null; no fabricated measurements.
Persistence	Store derived features + model/version + confidence; raw image retention is separate user preference.	Analysis-only mode deletes raw upload after feature extraction.
Progress	Compare normalized feature vectors and standardized photos over time.	Show confidence and capture quality before trend interpretation.

CV engineering checklist

- Create CVProvider interface so MediaPipe can be replaced/tested independently.
- Maintain test set across body shapes, skin tones, lighting, clothing coverage, camera angles and occlusions.
- Add confidence threshold and “insufficient evidence” result.
- Log model version and feature definition version with every scan.
- Do not store face embeddings or perform identity recognition.
- Document known limitations directly in the scan result UI.

08. Phase 3 - Nutrition Catalogue, Search and Daily Diet Tracker

Catalogue strategy

External datasets feed AthleteOS ingestion pipelines. Users search a normalized internal catalogue; meals reference immutable food versions/serving definitions so historical macros do not silently change.

Component	Build tasks	V1 acceptance
Food schema	Food, alias, cuisine, category, serving, nutrient, recipe, recipe ingredient, source/version.	Supports 100 g, household servings, bowl/katori/cup/piece when curated.
Indian layer	Curate Hindi/English/transliterated aliases and common dishes: aloo matar, bhindi, rajma, chole, dal, poha, idli, paneer dishes, roti etc.	Top Indian test queries return sensible result in top 5.
Global layer	Normalize broader staple and packaged-food data sources.	Source attribution stored; duplicates merged through canonical mapping.
Search	Prefix + typo/fuzzy + aliases + cuisine/category boost + recent foods.	P95 search target under 300 ms for cached/normal queries.
Meal log	Breakfast/lunch/snacks/dinner/custom; serving quantity; edit/delete; copy previous meal.	Daily calories/macros recompute transactionally.
Water	Quick-add presets + custom amount; daily target.	Dashboard and history remain consistent across units.

Dataset preparation pipeline

- Raw source snapshot -> staging tables -> nutrient/unit normalization -> alias enrichment -> duplicate resolution -> quality checks -> publish version.
- Create manually reviewed “golden dishes” for Indian recipes with source notes and adjustable recipe variants.
- Keep recipe nutrition computed from ingredients and edible quantity; do not hard-code macro values in UI.
- Support user-created foods/recipes but isolate them from global catalogue until reviewed.

09. Phase 4 - Training Module and Workout Logging

Core domain model

Program -> training week -> day/session template -> exercise prescription -> live workout session -> performed sets -> derived metrics/PRs -> progression decision.

Capability	Implementation detail	Acceptance test
Exercise catalogue	Exercise, movement pattern, muscle groups, equipment, modality, video/image ref, regression/progression links.	Filter by equipment and training type.
Plan templates	Bodybuilding PPL/upper-lower etc.; calisthenics skill paths; athletic strength/power/conditioning; aesthetic hypertrophy emphasis.	Template generation is deterministic for same profile/version.
Day view	Day number/date, target sets x rep range, rest, notes, last session values.	Day 22 can be opened directly and shows correct prescription.
Set logging	Weight/resistance, reps, RIR/RPE, completed/skipped/warmup, notes.	Every set can be edited; session totals update.
Offline durability	Local session draft + outbox sync + idempotency key + server version.	Browser refresh/network loss does not erase completed sets.
Derived metrics	Volume, estimated strength trend, PRs, completion, rep-quality proxy from RIR.	Fixtures produce expected results exactly.
Progression	Rep-range/double-progression, load increment rules, deload/hold flags, calisthenics progression gates.	Reason shown to athlete; athlete can keep/override.

Workout safety/product constraints

- No automatic maximal-load prescription for new users.
- Exercise substitutions respect equipment and user limitation flags.
- Allow pain/discomfort flag that stops automatic progression and recommends professional guidance when appropriate.
- Keep recommendations phrased as fitness guidance, not medical rehabilitation.

10. Phase 5 - Habit Tracker and Streak Engine

The habit tracker should visually behave like a spreadsheet: rows are habits, columns are dates, cells are fast toggle targets. On desktop it supports month density; on mobile it uses horizontally scrollable date columns and a compact “today” mode.

Feature	Technical approach	UX requirement
Habit definition	Name, category, schedule rule, start/end, target type (boolean/count/duration), optional reminder.	Create/edit/disable without losing history.
Daily completion	Unique habit_id + local_date; value and completion state; idempotent upsert.	One tap/click toggles boolean cells.
Schedule-aware streak	Count only expected schedule dates; timezone comes from athlete profile.	Rest days do not break a 5-days/week habit.
Month grid	API returns row definitions + compact completion matrix.	Fast scroll; sticky habit names and date headers on wide screens.
Summary	Current streak, longest streak, completion rate, category adherence.	Numbers agree with raw calendar cells.

Edge cases to test

- Timezone change while traveling.
- Backdated completion and edit permissions.
- Habit paused mid-month.
- Custom schedule such as Mon/Wed/Fri.
- Leap day/month boundaries and daylight-saving zones even if initial audience is India.

11. Phase 6 - Reports, Growth Dashboard and Analytics

Analytics pipeline

Transactional records + domain events -> daily aggregates -> weekly/monthly summaries -> dashboard/report API. Raw history remains queryable for audit and recalculation.

Report area	Metrics	Visualization
Body	Weight, measurement deltas, CV proportion trend/confidence.	Line charts + before/after progress-photo selector.
Training	Sessions, adherence, volume, sets by muscle/pattern, rep/load trend, PRs.	Trend lines, bars, PR cards, muscle-focus summary.
Nutrition	Calories, protein/carbs/fat, water, target adherence, meal consistency.	Goal rings + weekly average/delta.
Habits	Completion rate, current/longest streak, category consistency.	Calendar heatmap/month grid + comparison cards.
Recovery	Sleep, soreness/readiness input, session RPE where collected.	Weekly averages and correlation hints with careful wording.

Calculation rules

- Daily aggregates keyed by athlete_id + local_date.
- Weekly boundaries based on athlete locale preference (default Monday start).
- Monthly comparisons use comparable completed days when current month is partial.
- Every aggregate stores algorithm/version when derived logic can change.
- Report API returns data + explanation metadata so UI can label definitions.

12. Phase 7 - Adaptive Personalization Engine

Signals and priority order

Priority	Signal	Use
1	Actual performance history	Primary driver for load/reps/exercise progression.
2	Adherence and missed sessions	Adjust plan complexity/frequency; do not punish streaks for programmed rest.
3	Recovery/readiness	Hold volume/intensity when recovery is consistently poor; user can override.
4	Goal + training type + equipment	Constrains exercise selection and progression style.
5	Body metrics/CV trends	Secondary context for aesthetic balance/progress visualization; not sole plan driver.

Rules engine first

- Represent rules as versioned policies with test fixtures: inputs -> decision -> reason code.
- Examples: add reps within range; increase load after top-range completion with reserve; hold after repeated misses; suggest deload after fatigue pattern; swap inaccessible equipment.
- Generate recommendations as proposals. User accepts, edits, delays or rejects; all outcomes are logged.
- Never silently rewrite historical plans. Create a new plan version/effective date.

Later ML layer

After sufficient usage data exists, ranking models can choose among safe candidate exercises or estimate probability of successful progression. The deterministic policy remains the guardrail and fallback. Online experiments require explicit metric definitions and rollback.

13. Admin and Operations Console

Module	Admin capabilities	Controls
Exercises	Create/edit/version exercises, muscle tags, equipment, progression links.	Audit log; soft delete; publishing state.
Food catalogue	Review duplicates, aliases, recipes, nutrient source versions.	Draft/review/publish workflow.
Plan templates	Edit template constraints and progression parameters.	Versioning; test athlete preview before publish.
Feature flags	Enable CV/adaptive/report features by environment/cohort.	Role-limited; change audit; emergency kill switch.
Support	Lookup user by internal ID/email, view non-sensitive diagnostics, revoke sessions, trigger account export/deletion workflow.	Sensitive media not visible by default.
Monitoring	Job failures, ingestion quality, API health, sync conflict rates.	Links to runbooks and incident status.

Admin roles

- Support Agent: account and session actions only.
- Content Editor: exercises/food/template drafts.
- Reviewer/Publisher: approve catalogue/template versions.
- Engineer/Ops: feature flags and operational diagnostics.
- Security/Admin: privileged role assignment, audit review, retention operations.

14. Database and API Implementation Sequence

Schema groups

Group	Key entities
Identity	users, auth_sessions, consents, account_actions, audit_events
Athlete	athlete_profiles, goals, body_metrics, measurements, readiness_entries, equipment_profiles
CV	body_scans, scan_features, scan_media, cv_model_versions
Nutrition	foods, food_aliases, nutrients, servings, recipes, recipe_ingredients, meal_logs, meal_items, water_logs
Training	exercises, exercise_variants, programs, program_versions, workout_days, prescriptions, sessions, performed_sets, personal_records
Habits	habits, habit_schedules, habit_entries, streak_snapshots
Analytics	domain_events, daily_aggregates, weekly_reports, monthly_reports, recommendation_events
Ops	feature_flags, ingestion_runs, background_jobs, notification_preferences

API rollout order

- 1. /auth, /me, /consents, /onboarding
- 2. /metrics, /measurements, /readiness, /body-scans
- 3. /foods/search, /meals, /water
- 4. /programs, /workouts/today, /sessions, /sets, /exercises
- 5. /habits, /habit-entries, /streaks
- 6. /reports/weekly, /reports/monthly, /progress
- 7. /recommendations, /plan-versions, /feedback
- 8. /admin/* and operational endpoints

Contract rule: generate TypeScript API types from OpenAPI in CI. Breaking API changes require explicit version/migration strategy rather than UI/backend drifting independently.

15. Concept 3 UI Implementation Plan

Design language: dark navy surfaces, vivid pink/orange/purple gradients, high-contrast metric cards, compact visualizations, energetic but controlled accent use. Accessibility and information hierarchy take priority over decorative gradients.

Page group	Reusable components	Key states
Auth/onboarding	AuthCard, Stepper, MetricInput, ChoiceCard, ConsentPanel.	Validation, save-in-progress, resume, network error.
Dashboard	MetricCard, ProgressRing, WorkoutCard, NutritionRing, HabitMiniGrid, TrendChart.	First-day empty, partial data, completed day.
Workout	DaySelector, ExerciseRow, SetLogger, RestTimer, PreviousPerformance, SessionSummary.	Offline, sync pending, edit set, skipped exercise, finished.
Nutrition	MacroSummary, FoodSearch, FoodResult, ServingPicker, MealSection, WaterQuickAdd.	No results, aliases, user food, edited serving.
Habits	HabitGrid, DateHeader, StickyRowLabel, StreakCard.	Month loading, schedule rest day, paused habit.
Progress	PeriodTabs, MetricTrend, ComparisonCard, PhotoCompare, PRTimeline.	Insufficient history, partial month, low-confidence scan.
Body scan	CaptureGuide, UploadPanel, QualityResult, FeatureSummary, PrivacyControls.	Permission denied, low quality, processing, analysis-only.

Responsive breakpoints

- Desktop: persistent left navigation + multi-column dashboard.
- Tablet: collapsible navigation; 2-column cards.
- Mobile: bottom navigation; single-column cards; workout inputs optimized for one-hand tapping.
- Workout logging and meal logging receive larger touch targets than analytics/configuration screens.

16. Testing, Quality Gates and Security

Test layer	Coverage target / focus	Gate
Unit	Domain calculations: progression, streaks, macros, aggregate comparisons, conversion helpers.	Required on every PR.
API integration	Authz, ownership checks, transactions, idempotency, migrations, search filters.	Required before staging promotion.
Frontend component	Form validation, state transitions, accessibility, responsive behavior.	Critical flows covered.
E2E	Register -> onboard -> log meal -> workout -> habit -> report.	Smoke suite on staging and production candidate.
Offline/sync	Drop network mid-session, refresh, duplicate submit, conflicting edit.	No lost sets; duplicate-safe.
CV validation	Quality gate, landmark missingness, confidence, diverse test set, version regression.	No release on confidence regression beyond threshold.
Performance	Food search, dashboard report, workout save, concurrent API baseline.	Meet agreed P95 budget.
Security	SAST/dependency scan, API authorization/BOLA tests, rate limiting, signed media URLs, secret scan.	Critical/high findings block release unless formally accepted.

Security implementation checklist

- Resource ownership enforced server-side for every athlete-owned object.
- Passwords/identity provider handled using proven auth primitives; session revocation and rate limiting.
- Object storage private by default; short-lived signed URLs; randomized keys; content-type/size validation.
- PII and media separated from general logs; request logs avoid meal notes/photo URLs where unnecessary.
- Account export/delete workflow; retention jobs; audit trail for privileged admin access.
- Database backups encrypted; restore test before production launch.

17. DevOps, Deployment and Observability

Area	Implementation
Environments	Local -> ephemeral/CI -> staging -> production. Separate DB/storage/secrets per environment.
Containers	API and worker images built from pinned dependencies; non-root runtime where practical.
Migrations	Backward-compatible schema changes; migration job before application promotion; rollback/run-forward documented.
CI	Lint, typecheck, unit, integration, security/dependency scan, image build, contract generation.
CD	Automatic staging; production promotion from signed/tagged release; smoke tests + rollback.
Monitoring	API latency/error rate, DB pool, job queue depth, food ingestion failures, sync conflict rate, CV failure/confidence distributions.
Alerting	User-impacting SLO alerts; background data jobs alert at lower urgency; runbook link in each alert.
Backups	Automated DB backups + object storage lifecycle; scheduled restore rehearsal.

Recommended operational dashboards

- Authentication success/failure and suspicious rate spikes.
- Workout save latency + offline sync backlog/conflicts.
- Food-search latency + zero-result rate by query.
- Daily aggregate/report job freshness.
- CV quality rejection, inference errors and confidence distribution by model version.
- Feature-flag cohort health and recommendation accept/reject rates.

18. Team, Ownership and Delivery Capacity

Role	Recommended allocation	Primary ownership
Product/PM	0.5-1 FTE	Scope, acceptance criteria, athlete feedback, release decisions.
Product designer	0.5-1 FTE	Concept 3 system, responsive flows, prototypes, UX QA.
Frontend engineer	1-2 FTE	Next.js/PWA, local workout state, charts, accessibility.
Backend engineer	1-2 FTE	FastAPI, schema, authz, workout/nutrition/habit domain, analytics.
ML/CV engineer	0.5-1 FTE	CV adapter, validation, feature versioning, later ranking/ML.
QA/SDET	0.5-1 FTE	E2E, mobile/browser matrix, offline/sync and regression automation.
DevOps/SRE	0.25-0.5 FTE	CI/CD, infra, secrets, monitoring, backups, release runbooks.

RACI shortcuts

- Product owns acceptance; engineering owns technical correctness; design owns UX quality; QA owns release evidence; security-sensitive exceptions require explicit approver.
- Catalogue content (foods/exercises/templates) needs a named reviewer separate from implementation author once the app reaches beta.

If only 2-3 developers are available

Keep the same dependency order but stretch the calendar. Do not parallelize CV, nutrition ingestion and adaptive AI at the cost of workout reliability. A strong first release with excellent logging and reports is more valuable than several half-working AI features.

19. Risks, Mitigations and Feature Flags

Risk	Impact	Mitigation / kill switch
Food data quality/duplicates	Incorrect macros, poor search trust.	Golden test set, source/version, manual review, recipe calculation, nutrition_external_ingest flag.
CV inconsistency	Misleading progress conclusions.	Capture quality gate, confidence, diverse validation, analysis-only wording, cv_scan flag.
Workout plan over-complexity	Low adherence.	Simple template baseline, schedule/equipment constraints, user override, staged adaptive logic.
Offline workout loss	Severe trust failure.	Local durable draft, idempotent sync, recovery tests, sync status UI.
Recommendation surprise	User distrust or unsafe progression.	Reason codes, proposal/accept model, max change constraints, adaptive_recommendations flag.
Analytics mismatch	Users see contradictory numbers.	Single backend aggregation definitions + reconciliation tests.
Privacy breach of photos	High-severity incident.	Private buckets, signed URLs, retention control, analysis-only deletion, least privilege.
Scope creep	Delayed V1.	Freeze V1 deferred list and release criteria; all new ideas go to post-V1 backlog.

20. Beta and Production Launch Checklist

Private beta entry criteria

- All onboarding paths tested on Chrome/Edge/Safari/major mobile viewport.
- At least one complete program for each training type is content-reviewed.
- Indian food golden-search corpus and serving calculations pass.
- Workout session recovery under network loss passes automated/manual test.
- Weekly/monthly reports reconcile against source logs for seeded test athletes.
- CV has confidence/quality UI and can be disabled with no impact on core flow.
- Support/admin has minimum user-diagnostic tooling.

Production V1 gate

Gate	Evidence required
Functionality	E2E release suite green; acceptance criteria signed off.
Data	Production migrations reviewed; backup + restore rehearsal complete.
Security	No unaccepted critical/high issues; authz/resource ownership matrix tested.
Performance	Key P95 budgets within targets under expected beta/V1 load.
Operations	Dashboards, alerts, on-call owner, runbooks and rollback procedure active.
Privacy	Consent, retention, export/delete workflows verified.
Product	Terms/privacy copy, help/support routes, known limitations and beta feedback loop ready.

First 30 days after launch

- Review search zero-result queries and expand aliases/Indian dish catalogue weekly.
- Track onboarding drop-off by step.
- Track workout session completion and sync errors.
- Review recommendation acceptance/rejection before making rules more aggressive.
- Review CV retake rate/confidence and disable weak feature outputs rather than forcing coverage.
- Conduct first restore rehearsal and dependency/security update cycle.

21. Post-V1 Roadmap

Priority	Feature	Prerequisite
P1	Wearable imports: steps, sleep, heart rate/readiness	Stable profile/time-series ingestion and consent model.
P1	Coach portal with athlete plan review	Plan versioning + secure role/relationship model.
P1	Exercise video/form analysis pilot	Dedicated capture UX, movement-specific labeled validation data.
P2	Food photo suggestion	Nutrition catalogue quality + human confirmation workflow.
P2	Generative assistant for explanations	Guardrailed retrieval over athlete data + action confirmation.
P2	Social challenges/leaderboards	Privacy controls + anti-cheat + moderation.
P3	Subscriptions/premium tiers	Entitlement service, billing integration, support policy.
P3	Native apps	API/PWA stability and demonstrated mobile engagement.

Do not rush into ML

The highest-value dataset is created by reliable daily use. Treat V1 as the data-quality foundation: accurate sets, reps, load, RIR/RPE, meals, habits, recovery and plan outcomes. Once this history is trustworthy, machine-learning personalization becomes far more defensible.

22. Technical Definition of Done

Category	A story/feature is done only when...
Code	Reviewed, typed/linted, no unresolved critical TODOs, domain logic unit-tested.
API/data	Authorization verified, migration included, validation/error contract documented, idempotency considered.
UX	Matches design system, responsive states, keyboard/focus/accessibility checks, loading/empty/error states.
Analytics	Required events/metrics emitted with schema validation; no sensitive payload leakage.
Reliability	Retry/offline/duplicate behavior defined where relevant.
Security/privacy	Least privilege, ownership checks, retention/consent impact reviewed.
Operations	Logs/metrics added for failure modes; runbook note for operationally significant feature.
QA	Acceptance tests passed in staging with seeded and realistic data.
Release	Feature flag/default behavior defined; rollback path known.

Final implementation sequence



Foundation -> Identity -> Athlete profile -> Metrics/CV -> Nutrition -> Training -> Habits -> Analytics -> Adaptive recommendations -> Hardening -> Beta -> Production.

This ordering preserves the core promise of AthleteOS: a user can always track training and progress even if optional AI features are unavailable, and every later intelligence feature is grounded in clean first-party athlete history.